Substrate-Aware AI Agents: Execution Context as a First-Class Input

Manu Agrawal
Independent Researcher
manuagrawal2013@gmail.com

Abstract

AI agents are increasingly asked to write and execute code in environments whose memory, time, runtime, and operational constraints materially shape what counts as a good solution. Yet those constraints are often absent from the agent's context. We call this gap *substrate blindness*. We test whether supplying an execution contract before code generation changes the computational plan an AI agent selects. Three provider-configured model IDs-Anthropic `claude-opus-5`, OpenAI `gpt-5.6-sol`, and Google `gemini-3.7-flash`-generate a pairwise-distance program either from the task alone or with a 128 MB RAM and 10-second wall-time contract. Across 15 fresh direct-API pairs, contract disclosure lowers measured peak process memory in 13 of 14 executable comparisons and lowers mean execution time in every model cohort. The generated programs adapt through resource-relevant choices in block sizing, precision handling, traversal, temporary-buffer strategy, and input mapping. Under a tighter 96 MB contract, correct-and-under-budget outcomes are 5/5 for GPT, 4/5 for Claude, and 3/5 for Gemini, versus blind-reference outcomes of 1/5, 0/5, and 0/5, respectively. The results establish a clear proof of concept: execution context changes the implementations AI agents generate before they execute, making substrate awareness a first-class input to agent planning.

1. Silicon blindness

An agent can receive a complete task specification and still lack the information needed to produce a suitable action. The missing information is often not about the task itself; it is about the environment in which the task must be carried out.

This matters because modern computation is executed under real operating contracts. Kubernetes uses CPU and memory requests to schedule workloads and enforces limits at runtime [1]. Cloud Run terminates instances that exceed their configured memory limit [2]. AWS Lambda couples configured memory, CPU resources, and duration-based billing [3]. Runtimes impose language and dependency compatibility; tools have latency, reliability, permission, and quota boundaries. These conditions determine whether a plan is merely plausible or actually deployable.

Autonomous agents increasingly operate in harnesses that generate code, execute it, inspect outcomes, and iterate across long-running workflows. In many such systems, the harness, scheduler, or deployment configuration defines or can expose relevant parts of the execution contract-including memory limits, runtime versions, timeouts, tool permissions, and quotas-even when that information is absent from the model's inference context. Supplying this context during planning lets an agent choose against the relevant operating envelope before it relies on runtime feedback.

We call the failure to condition a plan on this information **substrate blindness**. The proposition of this paper is direct: execution context is decision-relevant information, and agents should receive it before they select a computational plan.

We demonstrate the proposition through numerical code generation, where the chosen implementation and its outcome can both be inspected. The intervention is minimal in content but consequential in effect: it supplies an operating contract without prescribing an algorithm, fine-tuning a model, or waiting for a failed execution, and asks whether that information changes what the model chooses to build.

Our contributions are:

• **Concept.** We formulate substrate blindness as an information-asymmetry problem in agent planning: the task is visible to the agent, while the operational environment that defines solution suitability is not.

• **Demonstration.** We provide a controlled paired experiment across three provider-configured model cohorts showing that pre-execution RAM/time disclosure changes generated implementations and substantially improves observed resource-time profiles.

• **Evidence.** We preserve the generated programs, execution profiles, and a source-linked audit, revealing concrete adaptation in implementation choices rather than superficial budget acknowledgement.

• **Research direction.** We show how the same principle naturally extends from memory and time to software runtime, accelerators, tools, quota, reliability, and cost-the broader substrate-awareness agenda.

2. From task context to execution context

Task context answers *what* an agent should do. Execution context answers *where*, *with what resources*, and *under which operating contract* it must do it. A substrate-aware agent receives both while it is deciding what program or action to produce.

The present intervention isolates execution context: the prompt provides a RAM/time contract, but no algorithm, no block size, no data-type instruction, and no post-failure repair loop. Any change in the generated implementation is therefore an adaptation to the disclosed operating envelope rather than compliance with a handed-down solution.

Numerical code generation offers a particularly transparent first demonstration. The same task can be solved by implementations with very different allocation and execution behavior, and the generated source, numerical output, peak process memory, and wall time can be evaluated together.

3. Controlled demonstration

3.1 Task and conditions

The task loads `vectors.npy`, an 8,000 by 1,024 float32 matrix, computes the sum of all pairwise Euclidean distances, and prints `TOTAL_DIST:<value>`. Materializing an 8,000 by 8,000 float32 distance intermediate alone requires 256 MB; bounded block algorithms provide correct alternatives with much lower peak memory.

The fresh direct-API cohort contains 15 predeclared A/D pairs: five each for Anthropic `claude-opus-5` (Opus), OpenAI `gpt-5.6-sol`, and Google `gemini-3.7-flash` (Flash). These are intentionally diverse provider configurations, not tier-matched controls or provider-wide capability rankings.

• **Blind (A):** the task specification.

• **Substrate-aware (D):** the identical task plus `RAM limit: 128 MB` and `Execution time limit: 10.0 seconds`.

The experiment compares what the models generate under these two information conditions. It does not prescribe a preferred implementation.

3.2 Measurement

Each generated program executes in an isolated macOS subprocess with Python 3.9.6, NumPy 2.0.2, and pinned single-thread BLAS-related environment variables. We record numerical correctness, exit status, elapsed wall time, and operating-system peak resident memory (`RUSAGE_CHILDREN` MaxRSS). A result is counted as correct only when its reported total matches an independently computed numerical reference.

The paper uses measured peak process memory as its resource outcome. The 128 MB and 96 MB labels identify whether a correct execution's measured peak falls within the stated operating envelope. In the 96 MB Claude extension, two malformed provider responses were retained in the archive and replaced under a predeclared, identical-prompt rule before replacement generation.

One blind Claude program used Python 3.10-style union syntax and failed under the pinned Python 3.9.6 runtime. It is retained as a first-pass correctness failure; its continuous RSS and wall-time measurements do not enter executable-only means.

4. Results: context changes the plan

4.1 Memory and time improve together

The effect is clear across the fresh paired cohort. In 13 of 14 executable A/D comparisons, the substrate-aware program has lower measured peak memory. Mean wall time also falls in every cohort: 2.52x for Claude, 1.68x for GPT, and 3.09x for Gemini. The disclosed operating envelope guides models toward implementations that are simultaneously more memory-efficient and faster in this task.

Configured model ID	Blind mean MaxRSS	Substrate-aware mean MaxRSS	Executable-pair RSS direction	Blind mean wall time	Substrate-aware mean wall time	Correct / measured <=128 MB (blind -> aware)
`claude-opus-5`	256.48 MB*	107.82 MB	lower 4/4	0.9109 s*	0.3612 s	0/5 -> 5/5
`gpt-5.6-sol`	118.63 MB	64.61 MB	lower 4/5; higher 1/5	0.5507 s	0.3282 s	4/5 -> 5/5
`gemini-3.7-flash`	452.36 MB	158.16 MB	lower 5/5	1.0994 s	0.3561 s	0/5 -> 2/5

* The Claude blind continuous means use the four executable blind programs. The fifth blind program is retained as a runtime-compatibility failure in the correctness/threshold denominator.

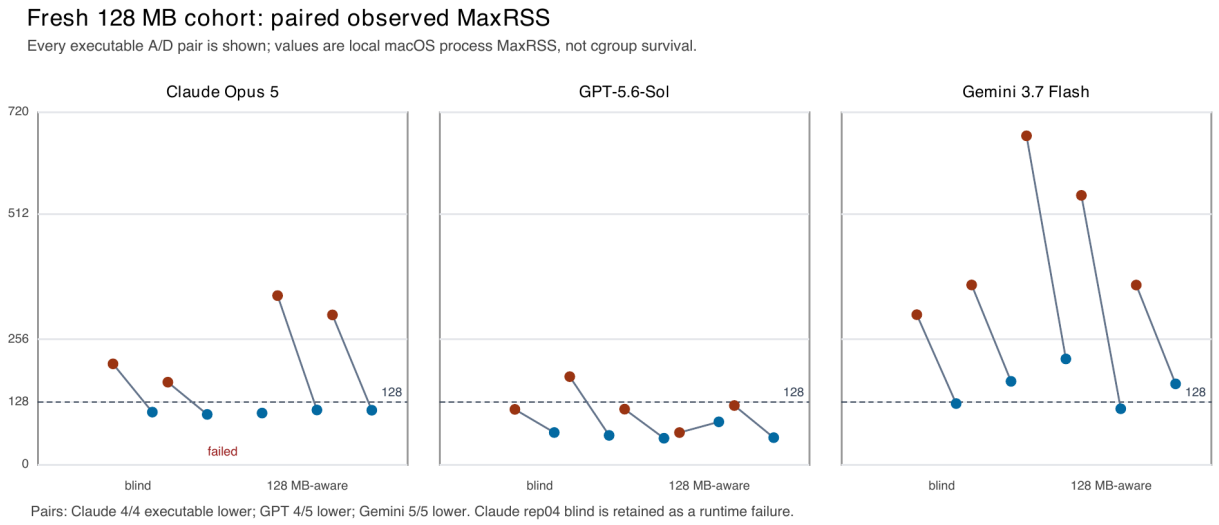

Figure 1. Each line connects an executable blind/substrate-aware pair. The figure makes the cohort-level result concrete while retaining the one GPT regression and the Claude runtime-compatibility failure.

4.2 How generated code adapts

The generated programs adapt at the level that matters: implementation choice. Across the audited corpus, the disclosed condition changes block sizing, precision handling, traversal extent, temporary-buffer strategy, and input mapping. These are the choices that determine how an otherwise correct numerical computation occupies memory and uses execution time.

The adaptation is not a single fixed recipe. Some blind programs already use blocking; some substrate-aware programs choose a different block geometry; others retain float32 in large intermediates, avoid a broad precision promotion, reuse a temporary array, or alter the portion of the pairwise matrix traversed. This is a strength of the result: execution context shifts the model's implementation distribution rather than forcing a single canned response.

The complete source-linked audit covers every included 128 MB script and every retained executable 96 MB script. It grounds the aggregate result in inspectable source while preserving the diversity of generated strategies.

4.3 Tighter contracts reveal graded responsiveness

We next supplied a 96 MB contract to five independently generated programs per model. This extension asks whether the stated envelope continues to shape generated implementations under a tighter boundary.

Configured model	Condition	Mean MaxRSS	RSS change vs blind	Mean wall time	Time change vs blind	Correct / <=96 MB
gpt-5.6-sol	Blind reference	118.63 MB	-	0.5507 s	-	1/5
	128 MB-aware reference	64.61 MB	-45.5%	0.3282 s	-40.4%	5/5
	96 MB-aware	60.88 MB	-48.7%	0.3582 s	-35.0%	5/5
claude-opus-5	Blind reference	256.48 MB*	-	0.9109 s*	-	0/5
	128 MB-aware reference	107.82 MB	-58.0%	0.3612 s	-60.4%	0/5
	96 MB-aware	87.57 MB	-65.9%	0.3802 s	-58.3%	4/5
gemini-3.7-flash	Blind reference	452.36 MB	-	1.0994 s	-	0/5
	128 MB-aware reference	158.16 MB	-65.0%	0.3561 s	-67.6%	0/5
	96 MB-aware	118.46 MB	-73.8%	0.3985 s	-63.8%	3/5

All 15 retained executable 96 MB programs are numerically correct and complete within the 10-second operating target. Relative to their blind references, the new 96 MB-aware cohorts lower both mean MaxRSS and mean wall time for every evaluated configuration. The effect is substantial for GPT as well as Claude and Gemini; the normalized view below makes this visible without allowing Gemini's larger absolute memory scale to compress the other model cohorts. Exact measured-budget fit remains model-dependent.

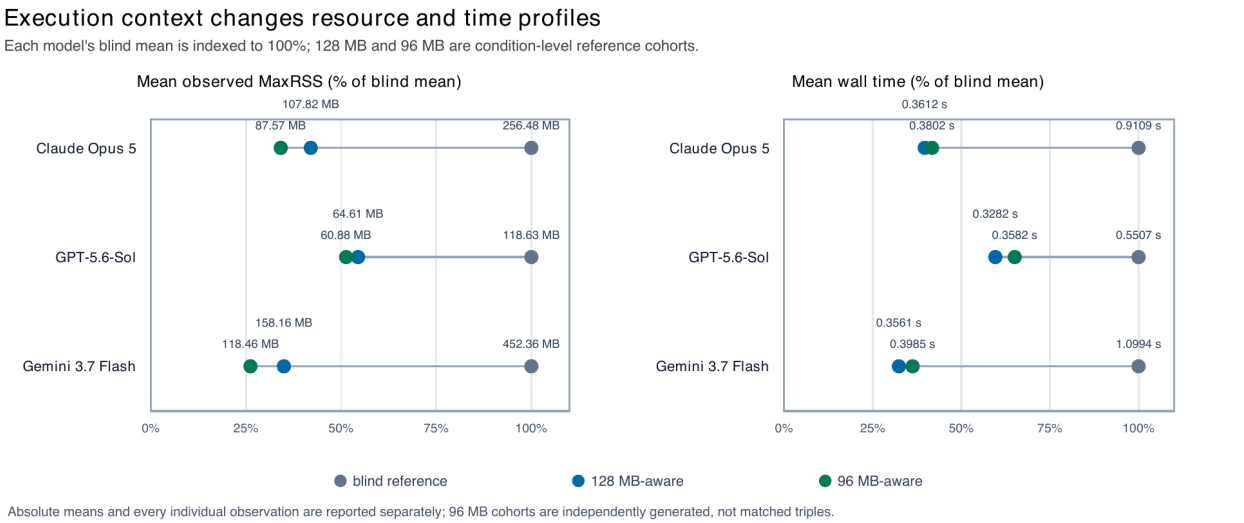

Figure 2. Each model's blind mean is indexed to 100%. Absolute means are shown next to each point and reported in Table 2. The 96 MB programs are independently sampled condition-level observations, not matched continuations of the reference cohorts.

Supplementary raw observations

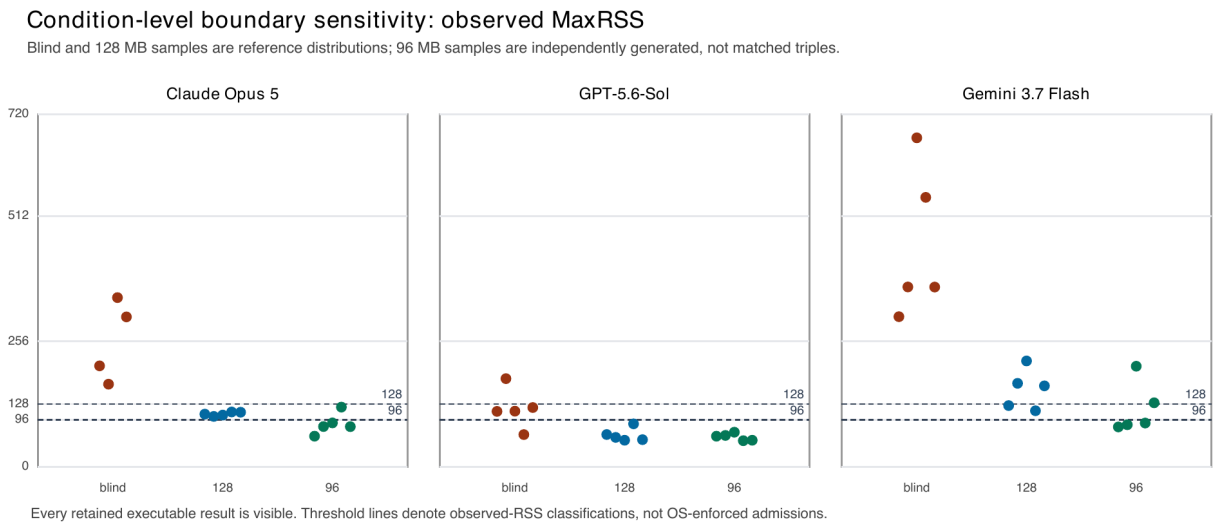

Figure S1. Every retained executable observation is visible. Blind and 128 MB aware results provide reference distributions; the 96 MB programs are independently sampled condition-level observations. This raw view preserves the complete distribution underlying the normalized comparison in Figure 2.

5. Related work

Language-model agents increasingly combine reasoning with actions in external environments. ReAct, for example, interleaves reasoning traces and task-specific actions, using environment interaction to update action plans [4]. Software-agent evaluation has likewise made execution environments central: SWE-bench evaluates whether models can resolve real repository issues that require coordination with a codebase and its tests [5].

This work studies a complementary moment in the agent lifecycle. Rather than providing execution feedback after an action fails, it supplies the relevant operating contract *before* an implementation is selected. The question is not whether execution feedback helps an agent repair a program; it is whether static execution context changes the program the agent chooses on its first generation. The paired design makes that earlier planning effect directly inspectable.

6. The broader substrate-awareness agenda

The central finding is consequential: execution context that materially determines plan suitability should be available to an agent during planning. Providing the operating contract before generation changes the computational implementation an agent selects. This is valuable even when a blind program happens to work under one environment, because suitability is defined by the environment in which deployment will actually occur.

The Python 3.9 runtime failure reported in Section 3.2 makes the same principle visible in software form. The controlled RAM/time experiment establishes the causal evidence in this paper; the failure shows that runtime version is another execution-context dimension that can determine whether generated code deploys successfully. Runtime version belongs alongside memory in the information an agent uses before it writes code.

The broader opportunity is substantial. A substrate-aware coding or tool-using agent can condition its plan on GPU memory, available CPU, runtime and dependency versions, tool latency, failure rates, permissions, quota, and cost. For agent harnesses, this makes pre-execution context a planning input rather than something the agent discovers only after a mismatch at runtime. The present result gives this agenda an empirical foundation: a minimal contract produces consequential changes in what the evaluated frontier configurations generate.

7. Research agenda and artifact availability

This paper establishes the first controlled demonstration in a numerical-code setting. The next studies extend the same intervention to runtime-version-aware generation, accelerator-aware multimodal computation, constrained data pipelines, and dynamic tool telemetry. Each will test the same core principle against the operating dimensions that matter for its setting.

The evaluation archive preserves the fresh direct-API manifest, prompts and dataset hashes, raw responses, generated scripts, numerical profiles, source-linked audit, and figure-generation code. The historical artifacts are retained for provenance and are not combined with the fresh cohort. A separately cleared public artifact release may be linked in a later version of this paper.

References

[1] Kubernetes Authors. *Resource Management for Pods and Containers*. Kubernetes documentation. https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/ (accessed 2026-08-27).

[2] Google Cloud. *Configure memory limits for services*. Cloud Run documentation. https://cloud.google.com/run/docs/configuring/services/memory-limits (accessed 2026-08-27).

[3] Amazon Web Services. *Configure Lambda function memory* and *AWS Lambda pricing*. https://docs.aws.amazon.com/lambda/latest/dg/configuration-memory.html and https://aws.amazon.com/lambda/pricing/ (accessed 2026-08-27).

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023. arXiv:2210.03629.

[5] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR 2024. arXiv:2310.06770.